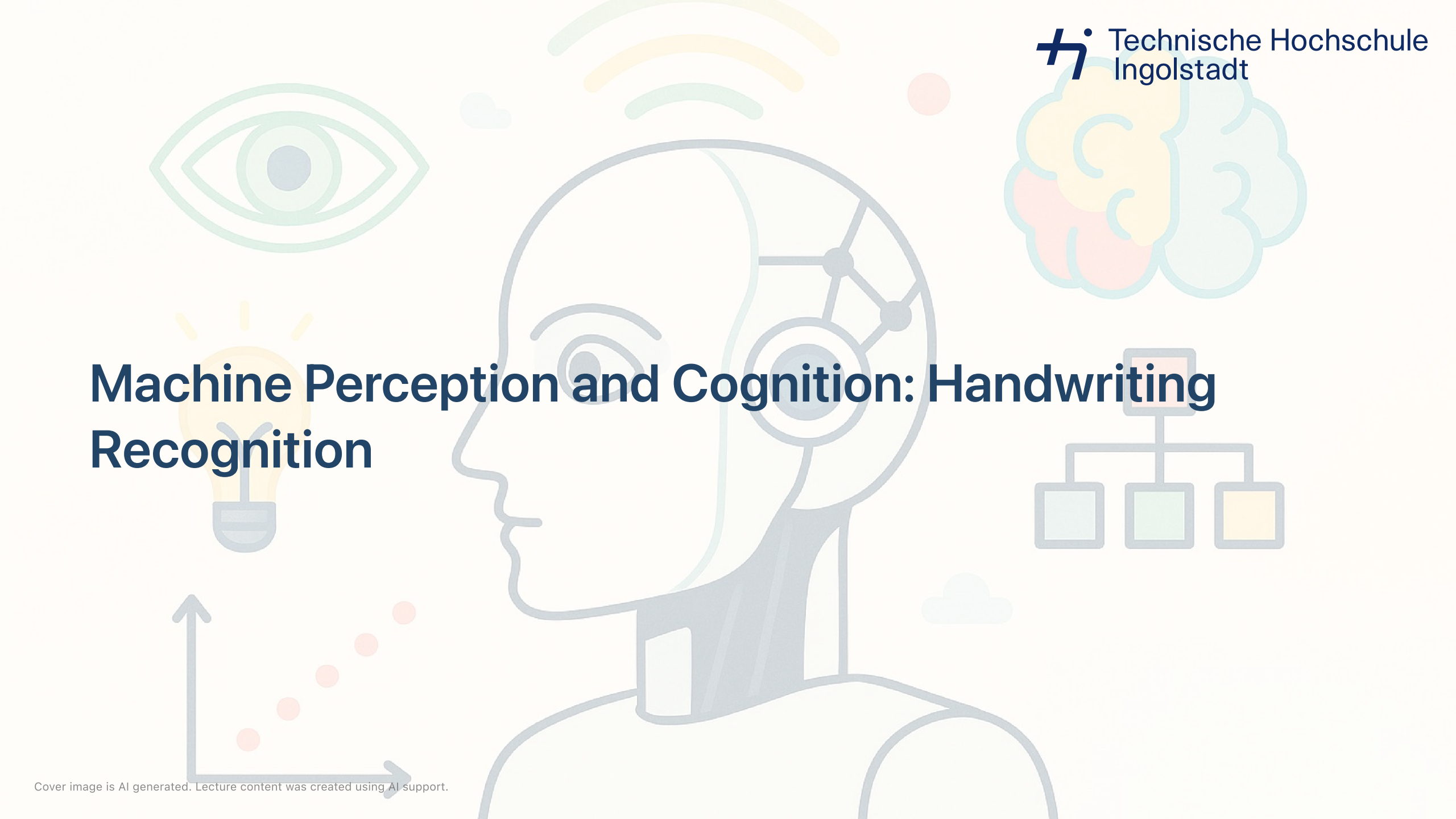
Technische Hochschule Ingolstadt

Machine Perception and Cognition: Handwriting Recognition



Cover image is AI generated. Lecture content was created using AI support.

What is Handwriting Recognition?

Handwriting Recognition (HWR) is the task of automatically converting handwritten text into a digital, machine-readable format.

Key applications:

- Digitization of historical documents and archives
- Postal address reading and sorting
- Form processing in banking and healthcare
- Real-time digital pen input (tablets, stylus)
- Ancient script deciphering and transcription

> The core challenge: handwriting varies enormously across writers, styles, languages, and recording conditions.

Online vs. Offline Handwriting Recognition

Two fundamentally different paradigms:

	Online	Offline
Input	Stroke sequences (x, y, t)	Static image
Acquired	During writing	After writing
Device	Digitizer, stylus	Scanner, camera
Temporal info	✓ Available	✗ Not available

Online Handwriting Recognition

The input is a **time series of pen positions** captured during writing.

- Pen position (x_t, y_t) sampled at regular intervals
- Additional signals: pen pressure, pen angle, pen-up/pen-down events
- Stroke: sequence of points between pen-down and pen-up
- Character: one or more strokes

Advantages:

- Order of strokes is known
- Temporal dynamics carry discriminative information

Example devices: graphics tablet, stylus, smart pens

Offline Handwriting Recognition

The input is a **static grayscale or binary image** of handwritten text.

Typical pipeline steps:

1. **Pre-processing:** binarization, skew correction, normalization
2. **Line segmentation:** isolate text lines from a document page
3. **Feature extraction:** convert image columns to feature vectors
4. **Recognition:** map feature sequence to character/word sequence
5. **Post-processing:** language model scoring, dictionary lookup

Challenge: No stroke order or temporal information is available.

The Segmentation Problem

A fundamental challenge in handwriting recognition is **segmentation**:

- Characters are **not uniformly spaced** in handwritten text
- Characters may be **connected** (cursive writing)
- A single character may span **multiple image columns**
- The same image column may contain parts of **two different characters**

Traditional approach: segment first, then recognize each segment.
→ Errors at each step compound: a bad segmentation leads to a wrong transcription.

Sayre's Knot

From Wikipedia, page "Sayre's paradox":

> Sayre's paradox is a dilemma encountered in the design of automated handwriting recognition systems. A standard statement of the paradox is that a cursively written word cannot be recognized without being segmented and cannot be segmented without being recognized. The paradox was first articulated in a 1973 publication by Kenneth M. Sayre, after whom it was named.

Historical Approaches to HWR

Before deep learning, the dominant approach used **Hidden Markov Models (HMMs)**:

1. Extract handcrafted features from image columns
2. Train one HMM per character class
3. Concatenate character HMMs to form word or line models
4. Viterbi decoding to find the most likely state sequence

Limitations of HMMs:

- Require explicit segmentation or complex topology design
- Generative model: not directly optimizing transcription accuracy
- Limited context
- Feature engineering is domain-specific

Why Neural Networks for Handwriting?

Neural networks offer several key advantages for HWR:

- **Automatic feature learning** — no handcrafted features needed
- **Discriminative training** — maximizes the conditional probability of the correct output sequence
- **Long-range context** — LSTMs capture dependencies across many timesteps
- **End-to-end training** — from raw image pixels to text, no segmentation needed

One landmark combination:

$$\text{Bidirectional LSTM} + \text{CTC} = \text{High-performance HWR}$$

Introduced by Graves et al. (2006, 2009) — still the foundation of many modern systems.

The Output Alphabet

In character-level handwriting recognition, the model predicts one **character class per timestep**.

A typical alphabet Σ for English may include:

- 26 lowercase letters: $a, b, c, \ldots, z$
- 26 uppercase letters: $A, B, C, \ldots, Z$
- 10 digits: $0, 1, \ldots, 9$
- Punctuation and special characters: `. , ; : ! ? ' " – ( )` ...
- Space character

The output layer of the network has $|\Sigma| + 1$ units — the additional one is for the **blank label**, introduced later.

One-Hot Encoding for the Alphabet

Each character in the alphabet is represented as a **one-hot vector**. For symbol k, define:

$$\mathbf{e}^{(k)} \in \{0,1\}^{|\Sigma|}, \quad e_j^{(k)} = \begin{cases} 1 & \text{if } j = k \\ 0 & \text{otherwise} \end{cases}$$

Example alphabet $\Sigma = \{a, b, c\}$:

Character	Index k	One-hot vector
a	1	$[1, 0, 0]$
b	2	$[0, 1, 0]$
c	3	$[0, 0, 1]$

One-Hot Encoding: Soft Predictions via Softmax

At each timestep t, the network outputs a **probability distribution** $\mathbf{y}_t \in [0,1]^{|\Sigma|+1}$:

$$\mathbf{y}_t = \text{softmax}(\mathbf{z}_t), \quad y_k^t = \frac{e^{z_k^t}}{\sum_j e^{z_j^t}}, \quad \sum_{k=1}^{|\Sigma|+1} y_k^t = 1$$

where $\mathbf{z}_t$ is the pre-activation (logit) vector from the network at time t.

- y_k^t: probability that the network assigns to character k at timestep t
- This is a **soft distribution** over all classes, not a one-hot vector

The Extended Alphabet: Adding the Blank Label

CTC introduces a special **blank label** ε (epsilon, sometimes written as $-$):

$$\Sigma' = \Sigma \cup \{\varepsilon\}$$

The blank label serves two essential purposes:

1. **Separator:** allows the model to output two consecutive identical characters.
    - Without blank: the path a, a, b merges to a, b → "ab"; there is **no way** to represent "aab" — any run of identical labels collapses to one
    - With blank: a, ε, a, b decodes to "aab" (blank separates the two a's, preventing merging); a, a, b still decodes to "ab"
2. **Filler:** allows the model to "say nothing" at timesteps that don't correspond to a new character output

The Extended Alphabet: Adding the Blank Label

The output layer has $|\Sigma'| = |\Sigma| + 1$ units, and the softmax normalizes over all of them.

What we need is a bidirectional LSTM that accepts handwritten text images as input and emits one softmax distribution over Σ' per sequence position.
The image width, or the width of the extracted feature map, is treated as the sequence length T.

Input Representation: The Feature Sequence

The handwritten text image is converted into a **sequence of feature vectors**:

1. **Column-wise features:** for each image column $t = 1, \ldots, T$:
    - Raw pixel intensities
    - Gradient features
    - Local vertical projection features
2. The result is a sequence $\mathbf{x} = (\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T)$

where $\mathbf{x}_t \in \mathbb{R}^d$ is the feature vector at column t.

Key property: $T \gg L$, where L is the number of characters — each character spans multiple columns.

Output: Label Sequences

The training target is a **label sequence** $\mathbf{l} = (l_1, l_2, \ldots, l_L)$ where $l_i \in \Sigma$:

- L = number of characters in the ground-truth transcription
- T = number of input timesteps (image columns)
- Necessary condition: $T \geq L$
- For repeated consecutive characters (e.g. "AA"), valid CTC alignments may require additional blank timesteps, so $T \geq L$ is **not always sufficient**
- In the worst case of all-adjacent repeats, at least $2L - 1$ timesteps are needed
- Example: for $L = 3$ and target "AAA", the shortest valid alignment is $A\varepsilon A\varepsilon A$, so CTC requires $T \geq 5$



Output: Label Sequences

The key challenge: **we do not know which columns correspond to which character**.

CTC learns this alignment **implicitly** from paired (image, transcription) data alone.

Image source: https://www.pexels.com/photo/question-mark-on-crumpled-paper-5428826/

Handwriting Recognition System

An overview over the pipeline from image to symbols using Connectionist Temporal Classification (CTC):

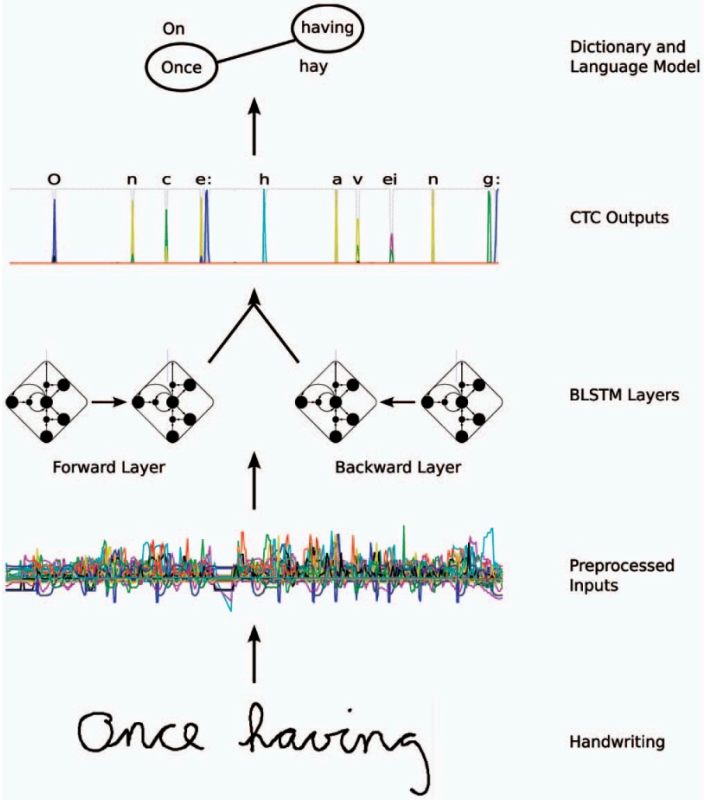


Image source: Graves, A., Liwicki, M., Fernández, S., Bertolami, R., Bunke, H., & Schmidhuber, J. (2009). A novel connectionist system for unconstrained handwriting recognition. IEEE transactions on pattern analysis and machine intelligence, 31(5), 855-868.

The Alignment Problem: Why CTC?

Traditional sequence labeling (e.g., frame-level cross-entropy) requires:

- **Pre-segmented** data: each input frame $\mathbf{x}_t$ labeled with its character
- Manual alignment or forced alignment

This is expensive, error-prone, and requires additional annotation tools.

CTC avoids this entirely: given only the input sequence $\mathbf{x}$ and the target string $\mathbf{l}$, CTC simultaneously learns:

- **What** characters to predict
- **When** to predict them
- **How long** each character persists in the latent alignment

> CTC enables end-to-end learning without any explicit per-frame alignment labels.

CTC: Core Idea

CTC works by summing over **all possible alignments** of a label sequence to an input sequence.

A **path** $\pi = (\pi_1, \pi_2, \ldots, \pi_T) \in \Sigma'^T$ is a sequence of labels (including blanks) of length T

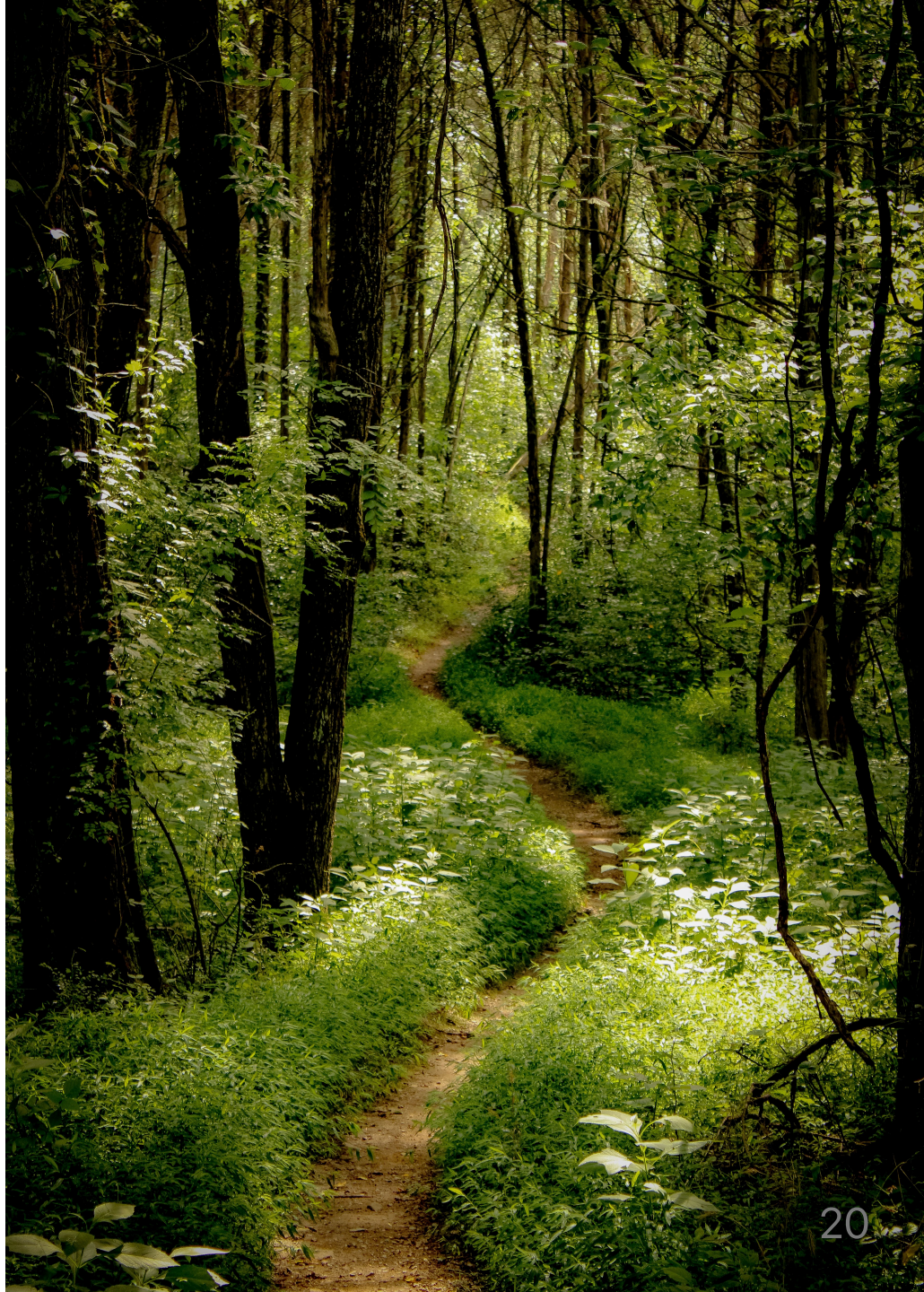


Image source: https://www.pexels.com/photo/serene-forest-path-under-lush-green-canopy-33081666/

CTC: Core Idea

Examples for target $\mathbf{l} = (C, A, T)$ with $T = 5$:

Path π	Decoded label
C, C, A, ε, T	CAT ✓
ε, C, A, A, T	CAT ✓
C, A, A, T, T	CAT ✓
$C, \varepsilon, A, \varepsilon, T$	CAT ✓

CTC sums the probabilities of **all such valid paths** that decode to the target.

Forward-Backward in CTC

Possible paths to align "CAT" over a sequence of length *T*:

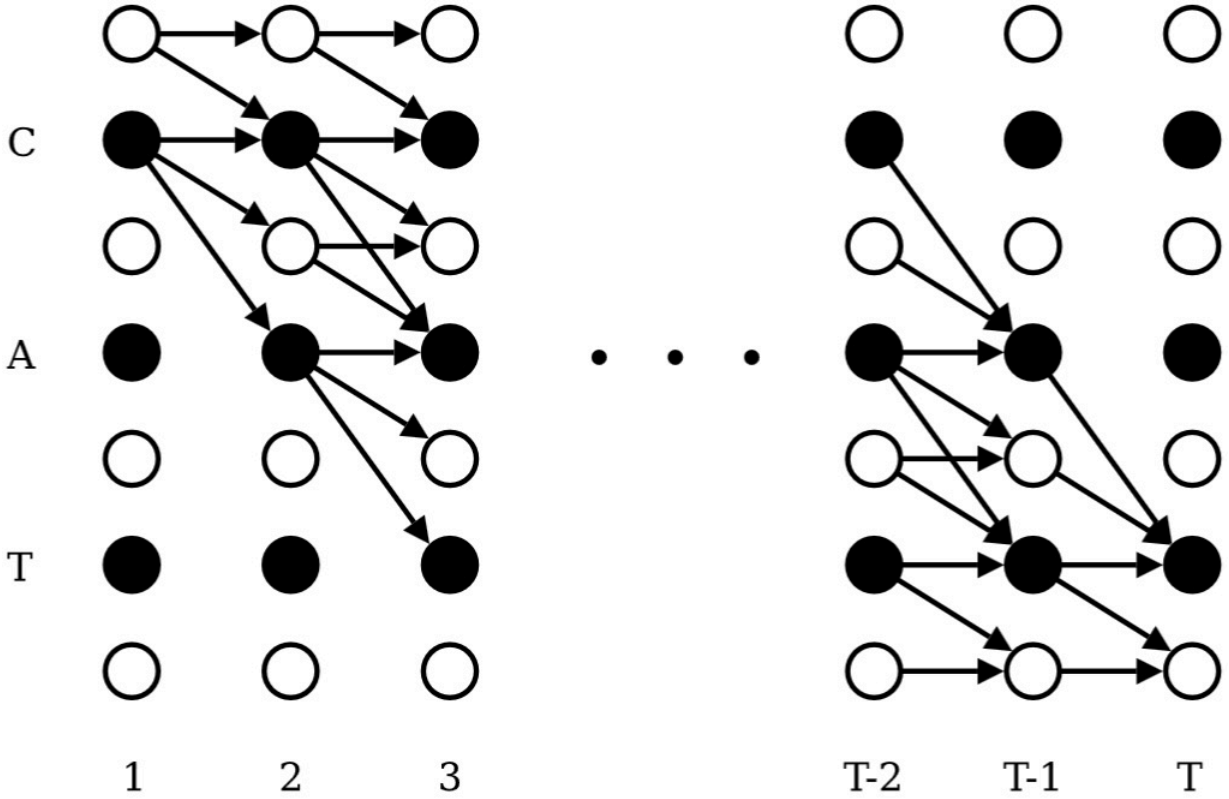


Image source: Graves, A., Fernández, S., Gomez, F., & Schmidhuber, J. (2006, June). Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks. In Proceedings of the 23rd international conference on Machine learning (pp. 369-376).

From Paths to Labels: The $\mathcal{B}$ Mapping

The mapping $\mathcal{B} : \Sigma'^T \to \Sigma^{\leq T}$ converts a path to a label sequence:

Step 1: Merge repeated consecutive labels

$$C, C, A, \varepsilon, T \xrightarrow{\text{merge}} C, A, \varepsilon, T$$

Step 2: Remove blank labels ε

$$C, A, \varepsilon, T \xrightarrow{\text{remove } \varepsilon} C, A, T$$

The blank is necessary to distinguish repeated characters:

- $\mathcal{B}(A, A, B) = AB$ (repeated A is merged)
- $\mathcal{B}(A, \varepsilon, A, B) = AAB$ (blank separates two As — they are not merged)

Example Handwritten Image

The next example is based on this original:



Image source: John Jackson F. E. I. S., Public domain, via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Salutation-Theory_and_Practice_of_Handwriting-010.jpg

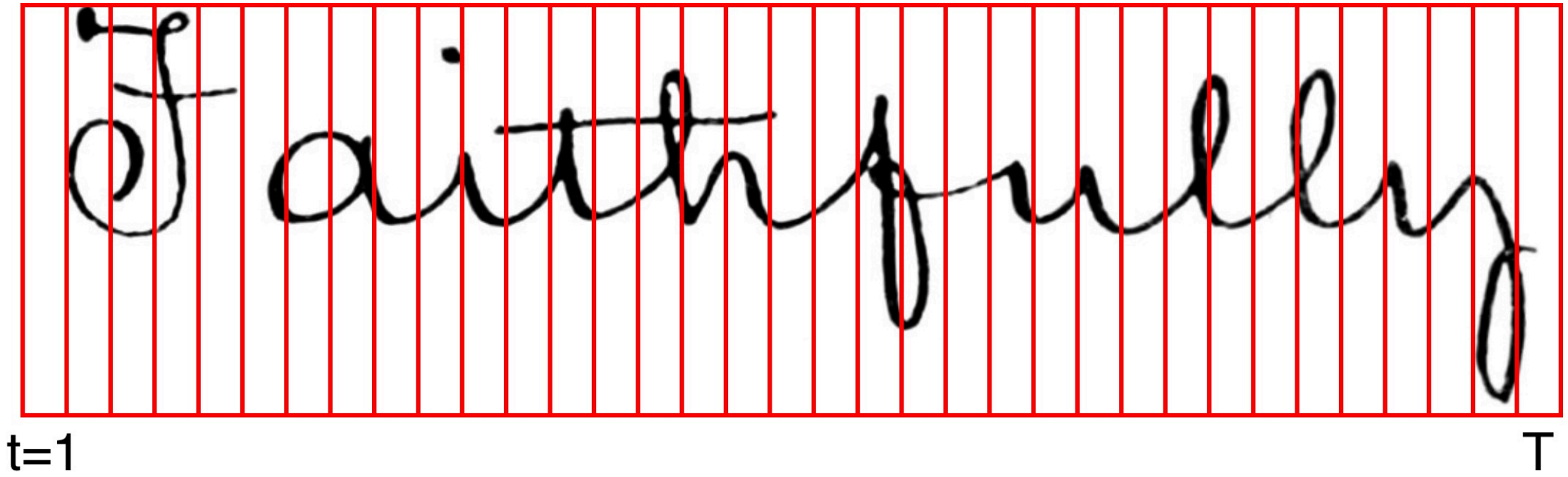
Example Handwritten Image

Cut-out of the first word:



Example Handwritten Image

Timesteps over the image width:



Here $T = 35$ and $\mathbf{l} = \text{Faithfully}$ with $L = 10$.

Example Handwritten Image

How can we write this down? Some example paths:

- $\pi = Faithfulɛlyɛɛɛ



From Paths to Labels: The $\mathcal{B}$ Mapping

The CTC probability of a label sequence $\mathbf{l}$ is:

$$p(\mathbf{l}|\mathbf{x}) = \sum_{\pi \in \mathcal{B}^{-1}(\mathbf{l})} p(\pi|\mathbf{x})$$

Image source: https://www.pexels.com/photo/still-life-with-dices-2073170/

Path Probability

Assuming **conditional independence** of outputs at each timestep:

$$p(\pi|\mathbf{x}) = \prod_{t=1}^{T} y_{\pi_t}^t$$

where y_k^t is the network's softmax output probability for label k at time t.

Path Probability

The probability of a label sequence is thus:

$$p(\mathbf{l}|\mathbf{x}) = \sum_{\pi \in \mathcal{B}^{-1}(\mathbf{l})} \prod_{t=1}^{T} y_{\pi_t}^t$$

Problem: the set $\mathcal{B}^{-1}(\mathbf{l})$ is exponentially large — it cannot be enumerated explicitly.

Solution: compute $p(\mathbf{l}|\mathbf{x})$ efficiently using a **dynamic programming forward-backward algorithm**.

Extended Label Sequence $\mathbf{l}'$

To handle blanks systematically, CTC works with an **extended label sequence $\mathbf{l}'$**:

Given $\mathbf{l} = (l_1, \ldots, l_L)$, insert blanks between every label and at the start and end:

$$\mathbf{l}' = (\varepsilon, l_1, \varepsilon, l_2, \varepsilon, \ldots, \varepsilon, l_L, \varepsilon)$$

Length: $|\mathbf{l}'| = 2L + 1$

Example:

$$\mathbf{l} = (C, A, T)$$
$$\mathbf{l}' = (\varepsilon, C, \varepsilon, A, \varepsilon, T, \varepsilon)$$

The forward-backward variables are defined over positions $s \in \{1, \ldots, |\mathbf{l}'|\}$ in $\mathbf{l}'$ and timesteps $t \in \{1, \ldots, T\}$.

CTC Forward Variable α

The **forward variable** $\alpha_t(s)$ is the total probability of all partial paths that, after t timesteps, end in trellis state s, which corresponds to position s in the extended label sequence $\mathbf{l}'$.

Note: odd positions in $\mathbf{l}'$ are blanks; even positions are characters. The index $\lfloor s/2 \rfloor$ maps from an extended-sequence position back to the corresponding character position in $\mathbf{l}$.

CTC Forward Variable α

Let $\mathcal{P}(s, t)$ be the set of valid partial paths of length t that end in state s of the CTC trellis.

$$\alpha_t(s) = \sum_{\pi_{1:t} \in \mathcal{P}(s,t)} \prod_{t'=1}^{t} y^{t'}_{\pi_{t'}}$$

Initialization at $t = 1$:

$$\alpha_1(1) = y^1_\varepsilon, \quad \alpha_1(2) = y^1_{l_1}, \quad \alpha_1(s) = 0 \text{ for } s > 2$$

The forward variable is computed via a **dynamic programming recurrence**.

Forward-Backward in CTC

Possible paths to align "CAT" over a sequence of length *T*:

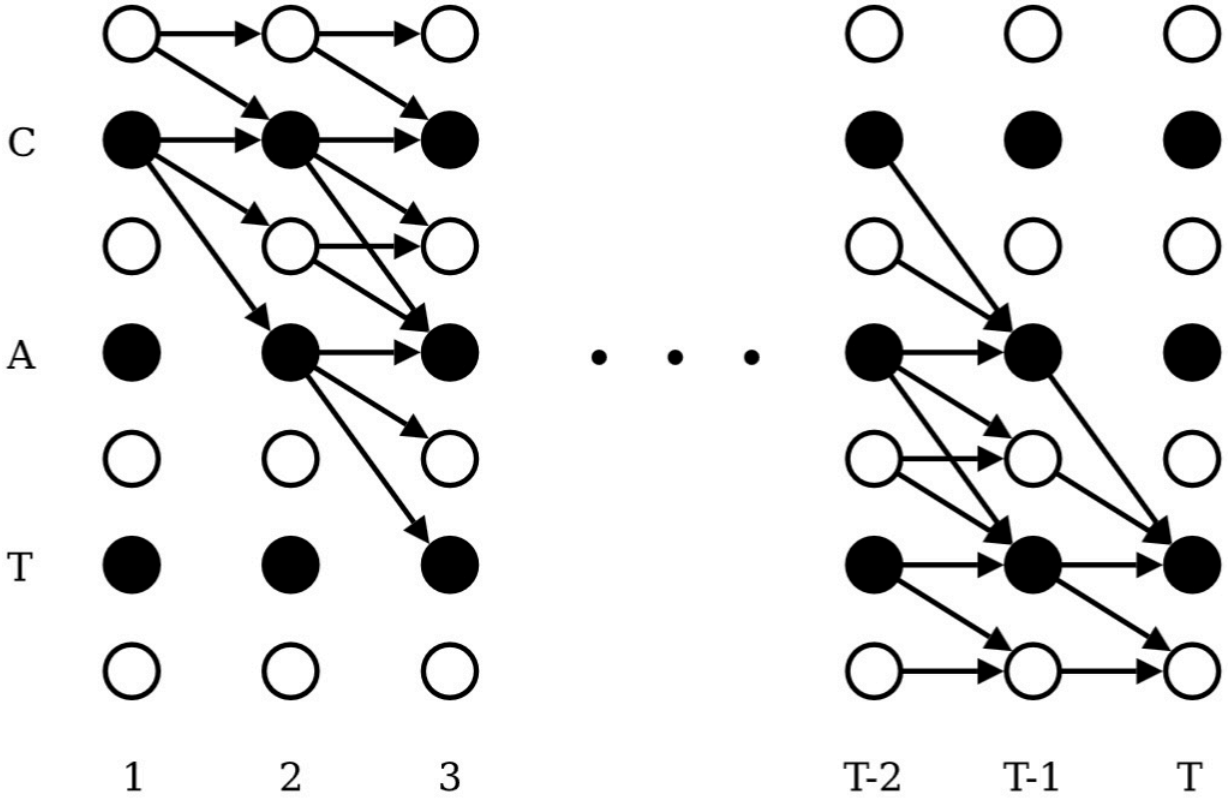


Image source: Graves, A., Fernández, S., Gomez, F., & Schmidhuber, J. (2006, June). Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks. In Proceedings of the 23rd international conference on Machine learning (pp. 369-376).

Forward Recurrence

At each timestep $t > 1$, the forward variable is updated as:

$$\alpha_t(s) = y^t_{\mathbf{l}'_s} \cdot \begin{cases} \alpha_{t-1}(s) + \alpha_{t-1}(s-1) & \text{if } \mathbf{l}'_s = \varepsilon \text{ or } \mathbf{l}'_s = \mathbf{l}'_{s-2} \\ \alpha_{t-1}(s) + \alpha_{t-1}(s-1) + \alpha_{t-1}(s-2) & \text{otherwise} \end{cases}$$

Three allowed transitions:

- **Stay** at the same position ($s \to s$): repeat a label or stay on blank
- **Advance by one** ($s - 1 \to s$): move from blank to character or from character to blank
- **Skip blank** ($s - 2 \to s$): jump from a character directly to the next character, bypassing the blank between them — only allowed when $\mathbf{l}'_s \neq \mathbf{l}'_{s-2}$, i.e., the two characters differ (skipping would conflate them if they are identical)

CTC Backward Variable β

The **backward variable** $\beta_t(s)$ accumulates the total probability of all valid suffix paths that start at state s at time t and continue to one of the two valid final states: the last label or the trailing blank.

In the convention used in this slide deck, $\beta_t(s)$ covers only the suffix from time $t+1$ onward and does **not** include the network output $y^t_{\mathbf{l}'_s}$ at the current timestep t.

CTC Backward Variable β

Initialization at $t = T$:

$$\beta_T(|\mathbf{l}'|) = 1, \quad \beta_T(|\mathbf{l}'| - 1) = 1, \quad \beta_T(s) = 0 \text{ for } s < |\mathbf{l}'| - 1$$

At $t = T$ the suffix from $t + 1$ onward is **empty** — there are no more timesteps to account for. The probability of an empty path is 1 (empty product), so the two valid terminal states are initialized to 1.

The backward recurrence mirrors the forward pass, but processes time in reverse.

Computing $p(\mathbf{l}|\mathbf{x})$

Using both forward and backward variables, the label sequence probability can be computed at **any** timestep t:

$$p(\mathbf{l}|\mathbf{x}) = \sum_{s=1}^{|\mathbf{l}'|} \alpha_t(s) \cdot \beta_t(s)$$

Note: $\alpha_t(s)$ includes the network output $y^t_{\mathbf{l}'_s}$ at time t, while $\beta_t(s)$ covers only the suffix from time $t+1$ onward and does **not** include $y^t_{\mathbf{l}'_s}$. Their product therefore gives the correct complete-path probability without double-counting.

In practice, the total probability is read off at $t = T$:

$$p(\mathbf{l}|\mathbf{x}) = \alpha_T(|\mathbf{l}'|) + \alpha_T(|\mathbf{l}'| - 1)$$

Computational cost: $O(T \cdot L)$ — linear in both sequence length and label length.

CTC Loss Function

CTC minimizes the **negative log-likelihood** over the training set $\mathcal{D} = \{(\mathbf{x}^{(i)}, \mathbf{l}^{(i)})\}$:

$$\mathcal{L}_{\text{CTC}} = -\sum_{i=1}^{|\mathcal{D}|} \log p(\mathbf{l}^{(i)} | \mathbf{x}^{(i)})$$

For a single training sample, the loss is:

$$\mathcal{L} = -\log p(\mathbf{l}|\mathbf{x}) = -\log \sum_{\pi \in \mathcal{B}^{-1}(\mathbf{l})} p(\pi|\mathbf{x})$$

In practice, computation is done in **log-space** to avoid numerical underflow for long sequences.

Natural Logarithm: $\log_e x$

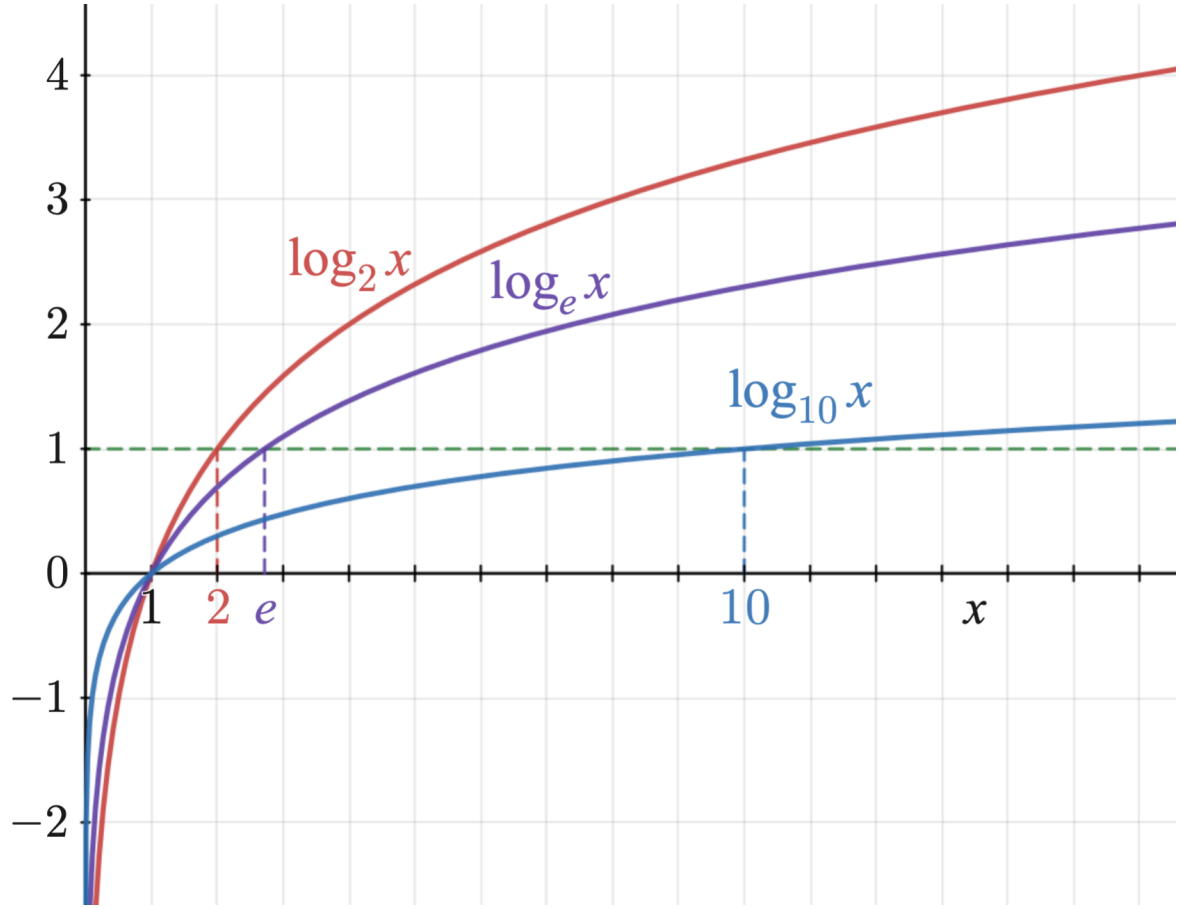


Image source: Richard F. Lyon, CC BY-SA 3.0 (https://creativecommons.org/licenses/by-sa/3.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Logarithm_plots.png

Gradient of CTC Loss

To train the network via backpropagation, we need the gradient $\frac{\partial \mathcal{L}}{\partial y_k^t}$:

$$\frac{\partial \mathcal{L}}{\partial y_k^t} = -\frac{1}{p(\mathbf{l}|\mathbf{x})} \sum_{s\,:\,\mathbf{l}'_s = k} \frac{\alpha_t(s) \cdot \beta_t(s)}{y_k^t}$$

Intuition: the gradient for output k at time t is large when the forward-backward product $\alpha_t(s) \cdot \beta_t(s)$ is large — meaning many paths pass through label k at time t when decoding $\mathbf{l}$.

This gradient is then passed through the **softmax** layer to get $\frac{\partial \mathcal{L}}{\partial z_k^t}$ and backpropagated through the BLSTM via standard BPTT.

Deriving $\frac{\partial \mathcal{L}}{\partial y_k^t}$: Step by Step

Start: $\mathcal{L} = -\log p(\mathbf{l}|\mathbf{x})$. Apply the chain rule of the logarithm:

$$\frac{\partial \mathcal{L}}{\partial y_k^t} = -\frac{1}{p(\mathbf{l}|\mathbf{x})} \cdot \frac{\partial p(\mathbf{l}|\mathbf{x})}{\partial y_k^t}$$

Deriving $\frac{\partial \mathcal{L}}{\partial y_k^t}$: Step by Step

Differentiate the path sum $p(\mathbf{l}|\mathbf{x}) = \sum_\pi \prod_{t'} y_{\pi_{t'}}^{t'}$. Because y_k^t appears only in the factor at time t:

$$\frac{\partial p(\pi|\mathbf{x})}{\partial y_k^t} = \begin{cases} \dfrac{p(\pi|\mathbf{x})}{y_k^t} & \text{if } \pi_t = k \\ 0 & \text{otherwise} \end{cases}$$

Summing over all paths that pass through label k at time t:

$$\frac{\partial p(\mathbf{l}|\mathbf{x})}{\partial y_k^t} = \frac{1}{y_k^t} \sum_{\substack{\pi \in \mathcal{B}^{-1}(\mathbf{l}) \\ \pi_t = k}} p(\pi|\mathbf{x})$$

Deriving $\frac{\partial \mathcal{L}}{\partial y_k^t}$: Step by Step

Connect to forward-backward variables. $\alpha_t(s)\,\beta_t(s)$ is exactly the probability of all complete paths that pass through trellis state s at time t (since $\alpha_t(s)$ accumulates all prefixes ending at s at time t — including the emission y_k^t — and $\beta_t(s)$ accumulates all suffixes from $t+1$ onward). Therefore:

$$\sum_{\substack{\pi \in \mathcal{B}^{-1}(\mathbf{l}) \\ \pi_t = k}} p(\pi|\mathbf{x}) = \sum_{s:\, \mathbf{l}'_s = k} \alpha_t(s)\,\beta_t(s)$$

Result:

$$\boxed{\frac{\partial \mathcal{L}}{\partial y_k^t} = -\frac{1}{p(\mathbf{l}|\mathbf{x})} \sum_{s:\, \mathbf{l}'_s = k} \frac{\alpha_t(s)\,\beta_t(s)}{y_k^t}}$$

Error Signal in CTC

Network prediction and error signal when training with CTC:

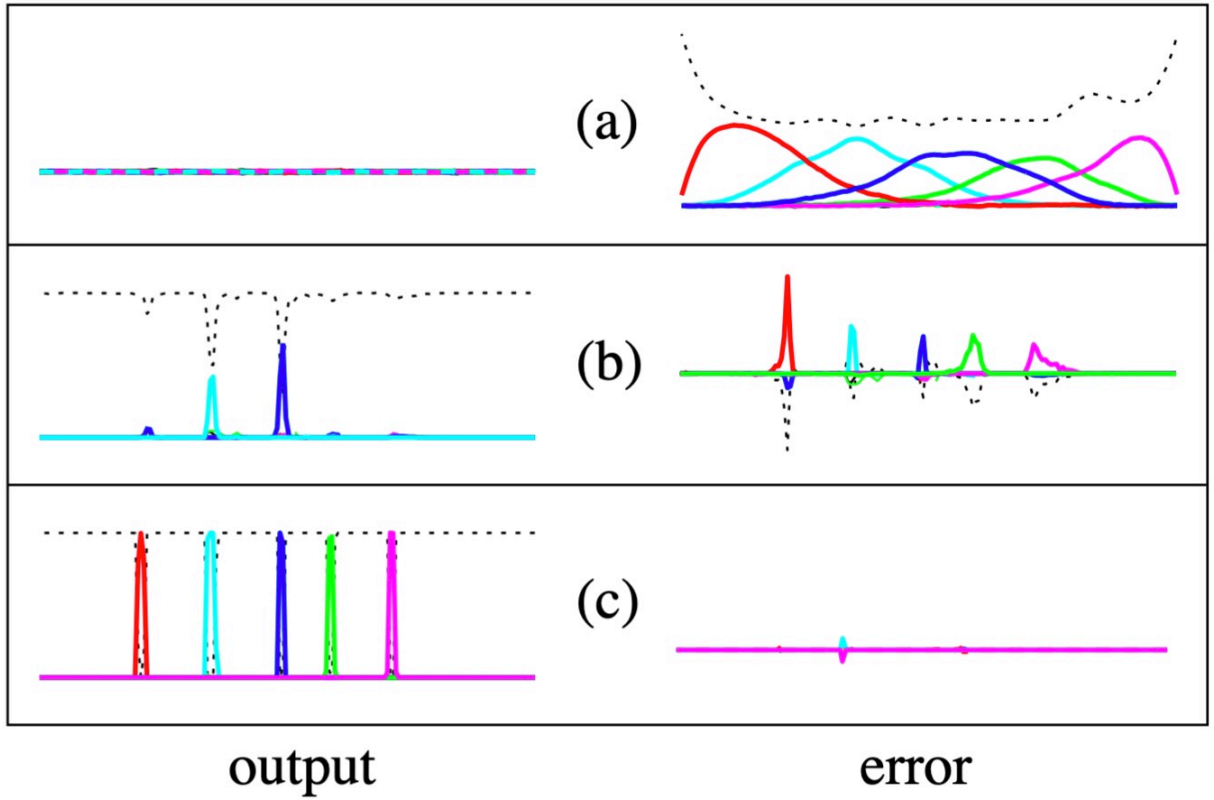


Image source: Graves, A., Fernández, S., Gomez, F., & Schmidhuber, J. (2006, June). Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks. In Proceedings of the 23rd international conference on Machine learning (pp. 369-376).

CTC vs. Frame-Level Cross-Entropy

Feature	CTC	Frame Cross-Entropy
Requires alignment labels	✕ No	✓ Yes
Training objective	Label sequence likelihood	Per-frame class accuracy
Output length	Variable	Fixed = input length
Error signal	Distributed over all paths	Per-frame, local
Handles repeated chars	✓ Via blank label	✕ Requires pre-processing
Suitable for cursive HWR	✓ Yes	Difficult



Decoding the CTC output

Image source: https://www.pexels.com/photo/photo-of-cryptic-character-codes-7319085/

Greedy (Best Path) Decoding

The simplest decoding strategy: at each timestep, take the **most probable label**:

$$\hat{\pi}_t = \arg\max_{k \in \Sigma'} y_k^t \quad \forall\, t = 1, \ldots, T$$

Then apply $\mathcal{B}$ to obtain the label sequence:

1. Merge repeated consecutive labels: $C, C, A, \varepsilon, T \to C, A, \varepsilon, T$
2. Remove blanks: $C, A, \varepsilon, T \to C, A, T$

Advantage: $O(T \cdot |\Sigma'|)$ — extremely fast, no memory overhead

Disadvantage: Ignores all other valid paths contributing to $p(\mathbf{l}|\mathbf{x})$

Limitations of Greedy Decoding

Example: Suppose the network could output "THE" or "TIE":

- Greedy selects the single path with the highest product of per-step probabilities
- But there may be **many more valid paths** mapping to "TIE" than to "THE"
- The **sum** of path probabilities for "TIE" may be larger than for "THE"
- → Greedy is **suboptimal** under the true CTC objective

The true CTC decoding criterion is:

$$\hat{\mathbf{l}} = \arg\max_{\mathbf{l}} \sum_{\pi \in \mathcal{B}^{-1}(\mathbf{l})} p(\pi|\mathbf{x})$$

This is NP-hard in general — we need approximate algorithms such as prefix search or beam search.

Prefix Search Decoding in CTC

Prefix search decoding in CTC:

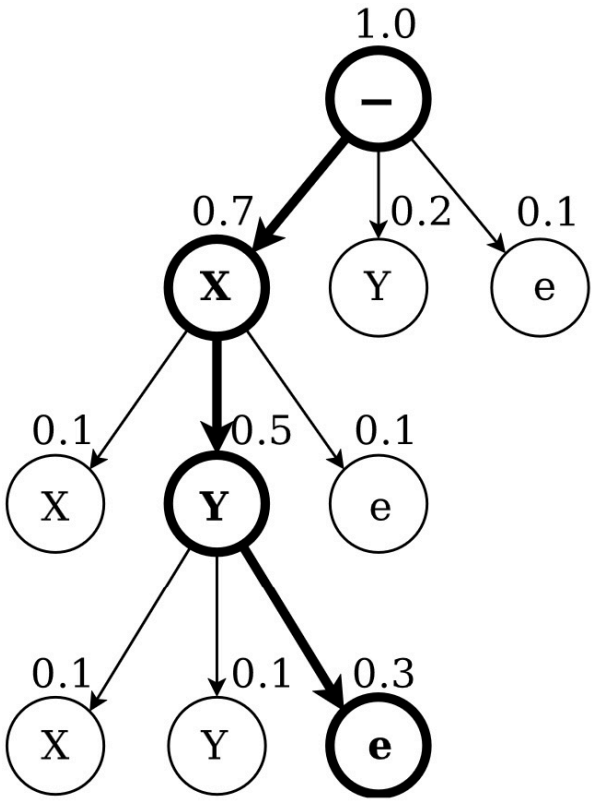


Image source: Graves, A., Fernández, S., Gomez, F., & Schmidhuber, J. (2006, June). Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks. In Proceedings of the 23rd international conference on Machine learning (pp. 369-376).

Prefix Search Decoding: Key Idea

Prefix search maintains a set of candidate label prefixes and their total CTC probabilities:

For each prefix $\mathbf{p}$, track two quantities at each time t (analogous to the forward variable α, but organized by label prefix rather than by position in the extended label sequence):

- $P_b(\mathbf{p}, t)$: total probability of all paths up to t ending in a **blank** that decode to $\mathbf{p}$
- $P_{nb}(\mathbf{p}, t)$: total probability of all paths up to t ending in a **non-blank** that decode to $\mathbf{p}$

Total probability: $P(\mathbf{p}, t) = P_b(\mathbf{p}, t) + P_{nb}(\mathbf{p}, t)$

These quantities are updated with efficient recurrences. Prefix-based decoding is substantially more accurate than greedy decoding, but it is also more expensive.

Beam Search Decoding

Beam search keeps the top-B most probable prefixes at each step:

1. Initialize with the empty prefix: beam $= \{()\}$ with probability 1
2. For each timestep $t = 1, \ldots, T$:
   - Extend each beam prefix with every label $k \in \Sigma'$
   - Compute the updated prefix probability using the CTC recurrences
   - Merge prefixes that decode to the same string
   - Keep only the top-B prefixes by total probability
3. Return the most probable complete label sequence in the beam

Complexity: $O(T \cdot B \cdot |\Sigma'|)$

Recurrent Neural Networks: Brief Recap

A **Recurrent Neural Network (RNN)** processes sequences by maintaining a hidden state:

$$\mathbf{h}_t = f(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b})$$

LSTM (Long Short-Term Memory) extends the RNN with gating mechanisms:

- **Forget gate** $\mathbf{f}_t$: decides what to erase from cell state
- **Input gate** $\mathbf{i}_t$: decides what new information to store
- **Output gate** $\mathbf{o}_t$: decides what to output from cell state
- **Cell state** $\mathbf{c}_t$: carries long-term memory across timesteps

LSTMs mitigate the **vanishing gradient** problem of plain RNNs, enabling learning of long-range dependencies that span hundreds of timesteps.

The Problem with Unidirectional RNNs

A standard (forward) LSTM at time t can only use **past context** $\mathbf{x}_1, \ldots, \mathbf{x}_t$:

$$\mathbf{h}_t^{\rightarrow} = \mathrm{LSTM}_{\mathrm{fwd}}(\mathbf{h}_{t-1}^{\rightarrow}, \mathbf{x}_t)$$

Problem for handwriting recognition:

- The correct interpretation of a character often depends on **future context**
- Example: "I" vs. "E" may only be distinguishable by what follows
- Example: "cl" vs. "d" depends on surrounding strokes

A unidirectional network is **causally limited** — it cannot access future information at training or inference time, making it systematically worse on ambiguous characters.

Bidirectional LSTM (BLSTM) Architecture

A **Bidirectional LSTM** uses two LSTM layers processing the sequence in opposite directions simultaneously:

$$\mathbf{h}_t^{\rightarrow} = \mathrm{LSTM}_{\mathrm{fwd}}(\mathbf{h}_{t-1}^{\rightarrow}, \mathbf{x}_t) \qquad (\text{left to right})$$

$$\mathbf{h}_t^{\leftarrow} = \mathrm{LSTM}_{\mathrm{bwd}}(\mathbf{h}_{t+1}^{\leftarrow}, \mathbf{x}_t) \qquad (\text{right to left})$$

The hidden states are **concatenated** at each timestep:

$$\mathbf{h}_t = [\mathbf{h}_t^{\rightarrow}; \mathbf{h}_t^{\leftarrow}]$$

At every position t, the combined state has access to the **full input sequence** — both past and future — enabling context-aware predictions across the entire sequence.

Why BLSTM for Handwriting Recognition?

In offline handwriting recognition, the **entire image is available at once** — there is no real-time constraint, so both directions can be processed freely.

The BLSTM leverages this by using:

- **Left context:** beginning of the word, preceding stroke patterns, writing style
- **Right context:** end of the word, confirming character identity

This enables resolution of **inherently ambiguous characters**:

- "rn" vs. "m": visually similar strokes — examining the spacing and what comes after reveals whether there are two letters or one
- "cl" vs. "d": past context (whether the strokes form a closed loop) and future context (letter spacing) both help disambiguate

BLSTM + CTC: The Full System

The complete end-to-end handwriting recognition pipeline:

1. **Feature extraction:** image → column-wise feature vectors $\mathbf{x}_1, \ldots, \mathbf{x}_T$
2. **Multiple BLSTM layers:** forward and backward LSTM pass over the sequence
3. **Output layer:** softmax over $|\Sigma'|$ classes at each timestep → $\mathbf{y}_t$
4. **CTC training:** maximize $p(\mathbf{l}|\mathbf{x})$ using the forward-backward algorithm
5. **Decoding:** greedy, prefix search, or beam search (+ optional language model)

Impact of Bidirectional LSTM on CTC

Phoneme transcription with CTC and using only a forward, backward or bidirectional LSTM:

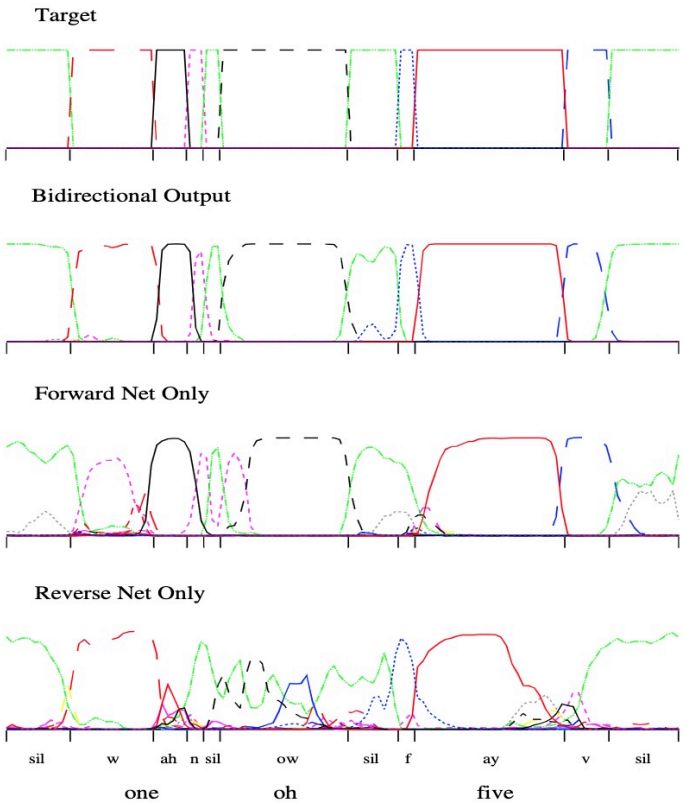


Image source: Graves, A., Fernández, S., & Schmidhuber, J. (2005, September). Bidirectional LSTM networks for improved phoneme classification and recognition. In International conference on artificial neural networks (pp. 799-804). Berlin, Heidelberg: Springer Berlin Heidelberg.

Modern Extensions: CNN + BLSTM + CTC

A widely used handwriting recognition architecture is a hybrid model:

1. **CNN layers:** extract local visual features from the image
    - Convolutional filters detect strokes, curves, junctions
    - Pooling reduces spatial resolution while preserving structure
2. **BLSTM layers:** model sequential dependencies over the CNN feature maps
3. **CTC output:** map the feature sequence to characters

Advantages over pure BLSTM:

- CNNs learn spatial features hierarchically
- Better invariance to local distortions, pen width, slant
- Fewer parameters needed in the recurrent layers

Summary: Key Concepts

One-hot encoding: each character mapped to a binary indicator vector; network output is a soft probability distribution via softmax over $\Sigma' = \Sigma \cup \{\varepsilon\}$

Blank label ε: enables CTC to handle variable-length alignment and repeated characters without explicit segmentation

CTC forward-backward: efficient dynamic programming over a trellis of (t, s) pairs; computes $p(\mathbf{l}|\mathbf{x})$ in $O(T \cdot L)$

CTC loss: negative log-likelihood; gradient involves the forward-backward product $\alpha_t(s) \cdot \beta_t(s)$

Decoding: greedy (fast), beam search (more accurate approximate decoding)

Key Takeaways

1. **Handwriting recognition** is an inherently sequential prediction problem requiring alignment between variable-length image inputs and variable-length text outputs.
2. **CTC** elegantly solves the alignment problem by marginalizing over all valid paths — no per-frame labels needed.
3. **One-hot encoding** and the **blank label** form the output representation; the network emits a soft distribution at every timestep.
4. The **forward-backward algorithm** enables efficient exact computation of the CTC gradient — essential for end-to-end training.
5. **BLSTM** provides full-sequence bidirectional context, resolving ambiguities that a unidirectional network cannot.